



Universidad Internacional del Ecuador

**Selección del Programa a desarrollar / Generación de Diagramas
funcionales y Arquitectura de Software**

Actividad académica – Proyecto Integrador

Estudiante: Erick Pries

Carrera: Ingeniería en Ciber Seguridad

Materia: Lógica de programación I

Docente: Mgs. Mónica Patricia Salazar Tapia

Fecha: 26 de junio de 2026

1. Introducción

El presente documento describe el desarrollo del proyecto Juego de la Serpiente, realizado como parte de la asignatura Lógica de Programación I. El objetivo principal del proyecto es aplicar los contenidos aprendidos durante la materia mediante la construcción de un programa interactivo desarrollado en Python.

El Juego de la Serpiente fue seleccionado porque permite trabajar conceptos fundamentales de programación, como variables, tipos de datos, operadores, condicionales, ciclos, listas, funciones, algoritmos, diagramas de flujo y depuración. Además, permite aplicar una estructura modular que facilita la organización del código y la comprensión del sistema.

El desarrollo del proyecto incluyó el análisis del problema, la elaboración de diagramas funcionales y de arquitectura, la implementación del código fuente, la documentación del sistema y la preparación de archivos de apoyo para su ejecución. Como resultado, se obtuvo un prototipo funcional que permite controlar una serpiente, recolectar comida, aumentar el puntaje y finalizar la partida cuando ocurre una colisión.

2. Alcance del proyecto

El proyecto consiste en el desarrollo de un videojuego básico denominado Juego de la Serpiente, implementado en el lenguaje de programación Python utilizando la librería Pygame. El sistema permite al usuario controlar una serpiente dentro de un tablero, recolectar comida, aumentar su puntaje, crecer progresivamente y finalizar la partida cuando ocurre una colisión con los bordes del tablero o con el propio cuerpo de la serpiente.

El alcance del proyecto incluye el análisis del problema, el diseño funcional mediante diagramas, la definición de una arquitectura modular, la implementación del código fuente, la documentación del sistema y la elaboración de archivos de apoyo para su ejecución.

El sistema desarrollado incluye las siguientes funcionalidades principales:

- Inicio y ejecución de una partida.
- Movimiento de la serpiente mediante teclado.
- Generación aleatoria de comida.
- Validación para evitar que la comida aparezca sobre la serpiente.
- Crecimiento de la serpiente al comer.
- Sistema de puntaje.
- Detección de colisiones con paredes.
- Detección de colisiones con el propio cuerpo.

- Pantalla de Game Over.
- Reinicio de partida.
- Salida del juego.
- Organización modular del código.
- Diagramas funcionales y de arquitectura.
- Archivo README con información general del proyecto.

El proyecto no contempla en esta etapa funcionalidades avanzadas como conexión a internet, base de datos, multijugador, sistema de usuarios, niveles complejos, inteligencia artificial, tienda de objetos o publicación del juego en plataformas externas. Estas características podrían considerarse como mejoras futuras.

3. Descripción del problema

El problema que se desea resolver consiste en desarrollar un programa interactivo que permita aplicar los conceptos fundamentales de lógica de programación mediante un videojuego sencillo. Para ello, se seleccionó el Juego de la Serpiente, ya que permite representar de manera práctica temas como variables, condicionales, ciclos, listas, funciones, eventos del teclado, detección de colisiones, manejo de estados y organización modular del código.

El juego plantea un escenario en el que el usuario controla una serpiente que se mueve dentro de un tablero. El objetivo del jugador es dirigir la serpiente hacia la comida que aparece en posiciones aleatorias. Cada vez que la serpiente come, aumenta su tamaño y se incrementa el puntaje. Sin embargo, el jugador debe evitar que la serpiente choque contra los bordes del tablero o contra su propio cuerpo, ya que esto provoca el fin de la partida.

Desde el punto de vista de programación, el problema requiere resolver varios aspectos: controlar el movimiento continuo de la serpiente, procesar entradas del teclado, almacenar las posiciones del cuerpo, generar comida aleatoria, validar colisiones, actualizar el puntaje y manejar los diferentes estados del juego.

Por lo tanto, este proyecto permite transformar un problema lógico en una solución computacional funcional, aplicando los contenidos revisados en la asignatura de Lógica de Programación I.

4. Relación con los contenidos de la asignatura

El proyecto se relaciona directamente con los contenidos revisados en la asignatura de Lógica de Programación I, ya que el desarrollo del Juego de la Serpiente requiere aplicar diferentes conceptos fundamentales.

Problemas y pasos para resolver problemas: El proyecto parte de la identificación de un problema: desarrollar un juego interactivo donde el usuario controle una serpiente dentro de un tablero. Para

resolverlo, se aplicaron pasos como comprender el problema, analizar los requerimientos, diseñar una solución, implementar el código, probar su funcionamiento y corregir errores.

Entornos de programación: El proyecto fue desarrollado utilizando Visual Studio Code como entorno de programación. Además, se utilizó Python como lenguaje principal y Pygame como librería para crear la ventana, dibujar elementos y manejar eventos del teclado.

Depuración: Durante el desarrollo se realizaron pruebas para identificar errores. Por ejemplo, se detectó un problema cuando el usuario cambiaba de dirección demasiado rápido. Para corregirlo, se implementó un sistema de input buffer que permite guardar direcciones válidas y evitar colisiones falsas.

Tipos de datos: El proyecto utiliza diferentes tipos de datos, como enteros para el puntaje, cadenas de texto para las direcciones, tuplas para representar coordenadas y booleanos para controlar condiciones como crecimiento o colisiones.

Manejo de datos, almacenamiento de datos y variables: El sistema almacena información en variables y atributos de clase. Por ejemplo, la posición de la serpiente se almacena en una lista, el puntaje se almacena en una variable numérica y el estado del juego se maneja mediante una enumeración.

Operadores y prioridad de operaciones: El proyecto utiliza operadores aritméticos para calcular posiciones, operadores de comparación para validar colisiones y operadores lógicos para combinar condiciones. Por ejemplo, se verifica si la cabeza de la serpiente está fuera de los límites del tablero mediante comparaciones como `head_x < 0` o `head_x >= SCREEN_WIDTH`.

Algoritmos y diagramas de flujo: Antes de programar, se diseñó un diagrama de flujo funcional que representa el algoritmo principal del juego. Este algoritmo muestra el proceso de iniciar la partida, mover la serpiente, detectar comida, actualizar puntaje, verificar colisiones y finalizar o reiniciar el juego.

Condicionales: Los condicionales se utilizan para tomar decisiones dentro del programa. Por ejemplo, el sistema verifica si la serpiente comió, si chocó con una pared, si chocó consigo misma, si el jugador presionó una tecla de dirección o si desea reiniciar la partida.

Ciclo while: El ciclo principal del juego se ejecuta mediante un while, que mantiene activo el programa mientras el estado del juego no sea EXIT. Dentro de este ciclo se procesan eventos, se actualiza la lógica y se dibuja la pantalla.

Ciclo for: Los ciclos for se utilizan para recorrer elementos, como los segmentos del cuerpo de la serpiente o las líneas de la cuadrícula. También se usan para dibujar cada parte de la serpiente en pantalla.

Tuplas: Las tuplas se utilizan para representar coordenadas dentro del tablero. Por ejemplo, una posición de la serpiente o de la comida se representa como (x, y).

Listas: La serpiente se representa mediante una lista de posiciones. Cada elemento de la lista corresponde a un segmento del cuerpo. La primera posición representa la cabeza y las demás representan el cuerpo.

Diccionarios: Los diccionarios se utilizan para relacionar direcciones opuestas. Por ejemplo, se define que "UP" es opuesto a "DOWN" y que "LEFT" es opuesto a "RIGHT". Esto permite evitar giros inválidos.

Funciones: El proyecto utiliza funciones y métodos para organizar el código. Por ejemplo, existen funciones para mover la serpiente, generar comida, verificar colisiones, aumentar el puntaje, dibujar elementos y reiniciar el juego.

5. Explicación del sistema desarrollado

El sistema desarrollado corresponde a una versión funcional del Juego de la Serpiente. El programa inicia creando una ventana gráfica con Pygame. Dentro de esta ventana se muestra un tablero dividido en celdas, una serpiente y una comida generada aleatoriamente.

El jugador controla la dirección de la serpiente mediante las flechas del teclado. La serpiente se mueve automáticamente en la dirección actual. Si la cabeza de la serpiente coincide con la posición de la comida, el sistema aumenta el puntaje, hace crecer la serpiente y genera una nueva comida en una posición válida.

El sistema también verifica constantemente si la serpiente choca contra los bordes del tablero o contra su propio cuerpo. Si esto ocurre, la partida termina y se muestra una pantalla de Game Over. Desde esta pantalla, el jugador puede reiniciar la partida presionando la tecla R o salir del juego presionando ESC.

El código fue organizado de forma modular. La configuración general se encuentra en ``settings.py``, el controlador principal en ``game.py``, la entrada del teclado en ``input_handler.py``, la serpiente en ``snake.py``, la comida en ``food.py``, las colisiones en ``collision_system.py``, el puntaje en ``score_system.py``, el renderizado en ``renderer.py`` y las pantallas especiales en ``screens.py``.

Esta organización facilita la comprensión del sistema, permite separar responsabilidades y hace posible ampliar el proyecto en futuras versiones.

6. Cronograma de actividades

Semana	Unidad / temas relacionados	Actividades del proyecto	Producto esperado
Semana 1	Unidad 1 – Tema 1 y Tema 2	Se inició el proyecto integrador mediante la selección del programa a desarrollar. Se analizaron diferentes opciones y se escogió el Juego de la Serpiente por ser un programa adecuado para aplicar lógica de programación, estructuras de control, variables, listas, funciones y diagramas. También se investigaron los tipos de diagramas funcionales y de arquitectura de software.	Selección del programa, justificación del proyecto e investigación inicial sobre diagramas.
Semana 2	Unidad 2 – Tema 3	Se inició el desarrollo del software mediante la configuración del entorno de programación. Se preparó la estructura inicial del proyecto, se instaló Python, Pygame y se configuró Visual Studio Code. Además, se empezó a organizar el proyecto en carpetas y módulos.	Entorno de desarrollo configurado y estructura inicial del proyecto.
Semana 3	Unidad 2 – Tema 4	Se elaboraron los diagramas funcionales y de arquitectura del sistema. Se desarrolló el diagrama de casos de uso, diagrama de flujo funcional, arquitectura por componentes y arquitectura por capas. Estos diagramas sirvieron como guía para el desarrollo del código.	Diagramas iniciales del sistema y planificación técnica del software.
Semana 4	Unidad 3 – Tema 5	Se inició el desarrollo del programa seleccionado. Se implementaron las primeras funcionalidades: creación de la ventana con Pygame, configuración general del juego, creación de la serpiente, movimiento básico y generación de comida.	Primera versión funcional básica del juego.

Semana 5	Unidad 3 – Tema 5	Se continuó con el desarrollo de funcionalidades lógicas. Se implementó el sistema de puntaje, crecimiento de la serpiente, detección de colisiones con paredes, detección de colisiones con el propio cuerpo y validación de comida. También se agregaron comentarios en partes importantes del código.	Prototipo funcional con lógica principal del juego.
Semana 6	Unidad 3 – Tema 6	Se aplicaron técnicas de depuración y mejora del programa. Se corrigieron errores detectados durante las pruebas, como el problema de giros rápidos de la serpiente. Para ello, se implementó un input buffer, permitiendo mejorar la fluidez del control. También se revisó la organización modular y se comentaron los archivos del proyecto.	Código depurado, comentado y mejorado.
Semana 7	Unidad 4 – Tema 7 y Tema 8	Se integró el producto final del proyecto. Se completó la documentación, el archivo README, los diagramas finales y el documento escrito. También se revisó que el repositorio contenga el código completo, la explicación de funcionalidades, el objetivo del sistema, los diagramas desarrollados y la fecha del proyecto. Finalmente, se preparó la entrega del software terminado.	Proyecto final integrado: código completo, diagramas, README, documento final y repositorio organizado.

7. Diagramas desarrollados

Como parte del diseño y documentación del proyecto Juego de la Serpiente, se elaboraron diferentes diagramas con el objetivo de representar visualmente el funcionamiento del sistema, la interacción del usuario con el programa y la organización interna del software. Estos diagramas permitieron planificar la solución antes de continuar con la implementación del código, facilitando la comprensión del problema y la forma en que cada módulo del sistema participa en el desarrollo del juego. Se pueden encontrar en el siguiente [enlace](#).

Los diagramas fueron desarrollados utilizando Mermaid, una herramienta que permite construir diagramas mediante texto estructurado. Esta herramienta resultó útil porque permite modificar,

reutilizar y mantener los diagramas de manera sencilla, además de facilitar su incorporación dentro de documentos técnicos o repositorios de software.

Los diagramas elaborados para el proyecto fueron los siguientes:

I. Diagrama funcional tipo casos de uso

El diagrama funcional tipo casos de uso representa las principales acciones que puede realizar el jugador dentro del sistema. En este diagrama, el actor principal es el Jugador, quien interactúa con el juego para iniciar una partida, controlar la dirección de la serpiente, visualizar el puntaje, reiniciar el juego o salir del programa.

Además, el diagrama muestra funcionalidades internas que el sistema debe ejecutar para que el juego funcione correctamente, como crear la serpiente inicial, generar comida, mover la serpiente, detectar si la comida fue alcanzada, actualizar el puntaje, verificar colisiones y finalizar la partida en caso de pérdida.

Este diagrama fue importante porque permitió identificar desde el inicio las funcionalidades principales del sistema y definir qué acciones debía permitir el programa desde la perspectiva del usuario.

II. Diagrama de flujo funcional

El diagrama de flujo funcional muestra la secuencia lógica del juego desde el inicio del programa hasta su finalización. En este flujo se representa cómo el sistema inicia, crea los elementos principales del juego, ejecuta el ciclo principal, procesa las entradas del teclado, mueve la serpiente y verifica las condiciones de comida o colisión.

El flujo inicia con la preparación de la partida, donde se crea la serpiente, se establece el puntaje inicial y se genera la comida. Luego, el programa entra en un ciclo principal en el que se leen las acciones del jugador, se actualiza la dirección de movimiento, se desplaza la serpiente y se comprueba si esta comió o chocó. Si la serpiente no colisiona, el juego continúa; si ocurre una colisión, se muestra la pantalla de Game Over y el jugador puede reiniciar o salir.

Este diagrama permitió ordenar el algoritmo general del juego antes de implementarlo en código.

III. Diagrama de arquitectura por componentes

El diagrama de arquitectura por componentes representa la división interna del sistema en módulos especializados. El proyecto fue organizado en componentes como la interfaz gráfica, el controlador principal del juego, el módulo de entrada de teclado, la lógica de la serpiente, el módulo de comida, el sistema de colisiones, el sistema de puntaje, el estado del juego y el renderizado.

Esta organización permite que cada componente tenga una responsabilidad específica dentro del programa. Por ejemplo, el archivo `game.py` coordina el ciclo principal del juego, `input_handler.py` procesa las teclas presionadas por el usuario, `snake.py` administra el movimiento de la serpiente,

food.py controla la comida, collision_system.py verifica las colisiones y renderer.py se encarga de dibujar los elementos visuales.

Este diagrama fue importante porque permitió relacionar el diseño del software con la estructura real del código fuente.

IV. Diagrama de arquitectura por capas

El diagrama de arquitectura por capas muestra el sistema desde una perspectiva más general, dividiendo el proyecto en niveles de responsabilidad. Las capas definidas fueron: capa de presentación, capa de control, capa de lógica del juego y capa de estado en memoria.

La capa de presentación se encarga de mostrar en pantalla el tablero, la serpiente, la comida, el puntaje y los mensajes del sistema. La capa de control coordina el ciclo principal del programa y comunica los diferentes módulos. La capa de lógica del juego contiene las reglas principales, como el movimiento, la generación de comida, la detección de colisiones y el aumento del puntaje. Finalmente, la capa de estado en memoria conserva la información actual del juego, como la posición de la serpiente, la dirección, el buffer de entrada, la comida, el puntaje y el estado de la partida.

Este diagrama permitió comprender cómo se organiza el programa desde una visión estructural y cómo las capas se comunican entre sí durante la ejecución del juego.

V. Diagrama de estados del juego

El diagrama de estados representa las diferentes situaciones en las que puede encontrarse el programa durante su ejecución. En la versión desarrollada, el juego maneja principalmente tres estados: RUNNING, GAME_OVER y EXIT.

El estado RUNNING indica que la partida está activa y que la serpiente puede moverse, comer y acumular puntos. El estado GAME_OVER se activa cuando la serpiente choca contra una pared o contra su propio cuerpo. Desde este estado, el jugador puede reiniciar la partida presionando la tecla R o salir del juego presionando ESC. Finalmente, el estado EXIT representa el cierre del programa.

Este diagrama fue útil para comprender cómo el sistema controla el flujo general de la partida y cómo responde ante las acciones del jugador o las condiciones del juego.

VI. Diagrama del input buffer

El diagrama del input buffer representa una mejora técnica implementada durante el desarrollo del proyecto. Este sistema permite guardar temporalmente las direcciones ingresadas rápidamente por el jugador y aplicarlas en orden durante los siguientes movimientos de la serpiente.

Esta mejora fue necesaria porque, durante las pruebas, se identificó que al presionar varias teclas rápidamente podían producirse movimientos no deseados o colisiones falsas. Con el input buffer,

el sistema valida si la dirección ingresada es válida, verifica que no sea una dirección repetida o contraria, y luego la agrega a una cola de movimientos pendientes.

Este diagrama demuestra cómo el proceso de depuración permitió mejorar la experiencia del usuario y hacer que el control de la serpiente sea más fluido y preciso.

En conjunto, los diagramas desarrollados permitieron documentar el proyecto desde diferentes perspectivas: funcional, lógica, arquitectónica y técnica. Además, evidencian que el desarrollo del juego no se realizó únicamente mediante programación directa, sino siguiendo un proceso de análisis, diseño, implementación y revisión.

8. Código completo del proyecto

Se encuentra en este [enlace](#).

9. Archivo README

Se encuentra en el siguiente [enlace](#).

10. Reflexión sobre el impacto de la tecnología

La tecnología tiene un impacto significativo en la forma en que se aprende, se resuelven problemas y se crean soluciones. A través de este proyecto fue posible comprobar que la programación no consiste únicamente en escribir código, sino en analizar un problema, dividirlo en partes más pequeñas, diseñar una solución lógica y luego implementarla de forma ordenada.

El desarrollo del Juego de la Serpiente demuestra cómo una idea sencilla puede convertirse en un sistema funcional mediante el uso de herramientas tecnológicas. Aunque se trata de un videojuego básico, su construcción requiere aplicar razonamiento lógico, organización, pruebas, depuración y toma de decisiones.

Además, este tipo de proyectos permite fortalecer habilidades importantes para el ámbito profesional, como la capacidad de resolver problemas, estructurar código, documentar procesos y mejorar una solución a partir de errores encontrados durante las pruebas.

En conclusión, la tecnología no solo permite crear programas, sino que también impulsa el pensamiento lógico, la creatividad y la capacidad de construir soluciones útiles a partir de problemas concretos.

11. Conclusión

El desarrollo del proyecto Juego de la Serpiente permitió aplicar de manera práctica los contenidos revisados en la asignatura de Lógica de Programación I. A través de este proyecto se pudo transformar un problema sencillo en una solución funcional, utilizando conceptos como variables, tipos de datos, operadores, condicionales, ciclos, listas, tuplas, diccionarios, funciones, diagramas de flujo, depuración y organización modular del código.

Durante el proceso de desarrollo se evidenció la importancia de analizar el problema antes de programar. La elaboración de diagramas funcionales y de arquitectura permitió comprender mejor el comportamiento esperado del sistema, identificar sus componentes principales y definir una estructura ordenada para el código. Esto facilitó la implementación del juego y permitió separar responsabilidades entre los diferentes módulos, como la serpiente, la comida, las colisiones, el puntaje, la entrada del teclado y el renderizado gráfico.

El sistema desarrollado cumple con el objetivo planteado, ya que permite al usuario controlar una serpiente dentro de un tablero, recolectar comida, aumentar el puntaje, crecer progresivamente y finalizar la partida cuando ocurre una colisión. Además, se incorporaron mejoras importantes durante la etapa de pruebas, como el sistema de input buffer, que permite mejorar la fluidez del control y evitar errores al presionar varias teclas rápidamente.

Este proyecto también permitió comprender que la programación no consiste únicamente en escribir instrucciones, sino en seguir un proceso lógico de resolución de problemas. Primero se identifica el problema, luego se analiza, se diseña una solución, se implementa el código, se prueba su funcionamiento y finalmente se corrigen errores. Este proceso fue fundamental para construir un sistema más ordenado, comprensible y fácil de mantener.

En conclusión, el Juego de la Serpiente representa una aplicación práctica de los fundamentos de la programación. Aunque se trata de un videojuego sencillo, su desarrollo permitió aplicar conocimientos esenciales para la construcción de software y reforzar habilidades como el pensamiento lógico, la organización, la depuración, la documentación y la capacidad de dividir un problema en partes más pequeñas para resolverlo de forma estructurada.